
✓ Practice Interview

Objective

The partner assignment aims to provide participants with the opportunity to practice coding in an interview context. You will analyze your partner's Assignment 1. Moreover, code reviews are common practice in a software development team. This assignment should give you a taste of the code review process.

Group Size

Each group should have 2 people. You will be assigned a partner

Part 1:

You and your partner must share each other's Assignment 1 submission.

✓ Part 2:

Create a Jupyter Notebook, create 6 of the following headings, and complete the following for your partner's assignment 1:

- Paraphrase the problem in your own words.

Your answer here

For Question 2, we have to return root(top) to leaf (bottom) paths of a binary tree in any order. The output will be a list of lists with each list consisting of integer values, where each value is a node's value in the order they appear in the path.

- Create 1 new example that demonstrates you understand the problem. Trace/walkthrough 1 example that your partner made and explain it.

Your answer here

Example:

Input: [10, 14, 9, 8, 16, 18]

10 is the root, 14 and 9 are left and right children of 10. 8, 16 are children of 14. 18 is child of 9.

Output: [[10, 14, 8], [10, 14, 16], [10, 9, 18]]

Example from partner assignment:

input = [4, 2, 6, 1, 3, 5, 7]

output = [[4, 2, 1], [4, 2, 3], [4, 6, 5], [4, 6, 7]]

In this example, 4 is the root, 2 and 6 are the left and right children of 4. 1, 3, 5, 7 are the children of 2 and 6. The output returns a list of lists where each list consists of node values in the order they appear in the path from the root to the leaf of the branch.

- Copy the solution your partner wrote.

Your answer here

```
from typing import List
```

```
class TreeNode:
```

```
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

```
def bt_path(root: TreeNode) -> List[List[int]]:
```

```
    if root is None:
        return []
```

```
    result = []
```

```
    def dfs(node: TreeNode, path: List[int]):
```

```
        if node is None:
            return
```

```
        # we add the current node to the path
        path.append(node.val)
```

```
        # if it's a leaf node, add the path to the result
```

```
        if node.left is None and node.right is None:
            result.append(path[:]) # Append a copy of the path
```

```
        # Recur for left and right children
```

```
        dfs(node.left, path)
```

```
        dfs(node.right, path)
```

```
    # Backtrack
```

```
# BACKTRACK
path.pop()

dfs(root, [])
return result
```

- Explain why their solution works in your own words.

Your answer here

This solution works because it uses depth-first search (DFS) to explore all paths from root to leaf. At each node, the function `dfs` recursively checks for the left and right children and builds a path from root to the current node. Also for each node, the value of current node is stored to path. If current node has no children, that means we have reached a leaf and the path is recorded as a root to leaf path and this complete path is appended to the result list. After the function has traversed the left and right subtrees of a node, it also uses backtracking to remove the current node from the path using the `pop` method. This ensures that the path list is reset when the function returns to the parent node to explore the other branches, to remove values from previously explored branches.

- Explain the problem's time and space complexity in your own words.

Your answer here

Each node is visited only once so the time complexity is $O(n)$, where n is the number of nodes. The space complexity is $O(h)$ as it depends on the depth of the recursion or the height of the tree.

- Critique your partner's solution, including explanation, and if there is anything that should be adjusted.

Your answer here

The solution is well-written and the code logic is also correct. Initially, I thought we can directly append path to result without copying it. However, considering the use of backtracking and how that alters the list, it makes sense to append copy of the path to the result to retain all the node values in the path. Overall the code is well-implemented and understandable with accurate use of helper function `dfs` and backtracking.

✓ Part 3:

Please write a 200 word reflection documenting your process from assignment 1, and your presentation and review experience with your partner at the bottom of the Jupyter Notebook under a new heading "Reflection." Again, export this Notebook as pdf.

✓ Reflection

Your answer here

In assignment 1, I worked on Question 1 where the task was to check for duplicates in a binary tree and to return that duplicate value. In case of multiple duplicates we return the duplicate closest to the root. For this, I used pre-order depth-first search (DFS) that allows us to visit each node in the tree recursively starting from the root node and then its left and right children, ensuring we return the closest duplicate to the root. I used a dictionary to store node values and their depths, to keep track of the nodes visited. When a duplicate is found, the function stops and returns the value.

In this assignment, I reviewed Question 2 done by my partner. In Question 2 we return the root to leaf paths in any order. After reviewing the solution for this question, I found it to be logically correct and well-implemented. The concept of backtracking was a bit confusing to me initially but going through some example cases helped me to understand the concept and the importance of storing a copy of the path list to 'result' to ensure backtracking does not change already stored paths. This assignment allowed me to work on two different questions that helped to develop knowledge on new concepts like DFS and backtracking.

Evaluation Criteria

We are looking for the similar points as Assignment 1

- Problem is accurately stated
- New example is correct and easily understandable
- Correctness, time, and space complexity of the coding solution
- Clarity in explaining why the solution works, its time and space complexity
- Quality of critique of your partner's assignment, if necessary

Submission Information

 **Please review our [Assignment Submission Guide](#)**  for detailed instructions on how to format, branch, and submit your work. Following these guidelines is crucial for your submissions to be evaluated correctly.

Submission Parameters:

- Submission Due Date: HH:MM AM/PM - DD/MM/YYYY
- The branch name for your repo should be: `assignment-2`
- What to submit for this assignment:
 - This Jupyter Notebook (`assignment_2.ipynb`) should be populated and should be the only change in your pull request.
- What the pull request link should look like for this assignment: `https://github.com/<your_github_username>/algorithms_and_data_structures/pull/<pr_id>`
 - Open a private window in your browser. Copy and paste the link to your pull request into the address bar. Make sure you can see your pull request properly. This helps the technical facilitator and learning support staff review your submission easily.

Checklist:

- ☐ Created a branch with the correct naming convention.
- ☐ Ensured that the repository is public.
- ☐ Reviewed the PR description guidelines and adhered to them.
- ☐ Verify that the link is accessible in a private browser window.

If you encounter any difficulties or have questions, please don't hesitate to reach out to our team via our Slack at `#cohort-3-help`. Our Technical Facilitators and Learning Support staff are here to help you navigate any challenges.

